

Biến đồ thị thành cây

make_pair(immortalCO, WrongAnswer)

11 tháng 3 năm 2017

Nguồn gốc: Making Graphs into Trees

Graph Theory / Making Graphs into Trees - immortalCO, WrongAnswer

Abstract

Trong nhiều bài toán đồ thị của OI, nếu có thể biến đồ thị thành một cấu trúc dạng cây thì lời giải trở nên thuận tiện hơn nhiều. Tài liệu gốc tổng kết ba cách nhìn chính: cây DFS, cây BFS và cây tròn-vuông của đồ thị cactus. Bản dịch chuyển bộ slide thành ghi chú LaTeX, giữ lại các định nghĩa, tính chất và ứng dụng thuật toán chính; các hình minh họa được diễn giải bằng lời.

Contents

1	Cây DFS	1
1.1	Xử lý cactus bằng cây DFS	1
1.2	Tập độc lập lớn nhất trên cactus	1
1.3	Quy hoạch động trạng thái nén trên cây DFS	2
2	Cây BFS	3
3	Cây tròn-vuông của cactus	3
3.1	Định nghĩa	3
3.2	Xây dựng	3
4	Ứng dụng của cây tròn-vuông	4
4.1	DP trên cactus	4
4.2	Truy vấn đường đi ngắn nhất	4
4.3	Virtual cactus	4
4.4	Phân rã và phân tách trên cactus	5
5	So sánh với cây co chu trình	5
6	Cây tròn-vuông tổng quát	6

1 Cây DFS

Cho đồ thị liên thông vô hướng $G = (V, E)$. Khi chạy DFS từ một gốc r , các cạnh được DFS đi qua tạo thành một cây

$$T = (V, E').$$

Ta gọi các cạnh trong E' là **cạnh cây**, còn các cạnh trong $E \setminus E'$ là **cạnh ngoài cây**.

Trong cây DFS của đồ thị vô hướng, mọi cạnh ngoài cây (u, v) đều nối một đỉnh với một tổ tiên của nó trong T . Với cạnh ngoài cây (u, v) , ta nói nó **phủ** các cạnh cây nằm trên đường đi giữa u và v trong T . Tính chất đơn giản này là lý do cây DFS thường biến các ràng buộc chu trình thành ràng buộc trên đường đi trong cây.

1.1 Xử lý cactus bằng cây DFS

Một cactus là đồ thị vô hướng liên thông trong đó mỗi cạnh thuộc nhiều nhất một chu trình đơn. Sau khi dựng cây DFS của cactus, mỗi cạnh ngoài cây tương ứng đúng với một chu trình: chu trình gồm chính cạnh ngoài cây đó và các cạnh cây mà nó phủ. Vì các chu trình của cactus không giao nhau theo cạnh, hai cạnh ngoài cây khác nhau sẽ phủ hai tập cạnh cây rời nhau. Nhờ đó ta có thể tìm thuận tiện mỗi cạnh thuộc chu trình nào.

1.2 Tập độc lập lớn nhất trên cactus

Bài toán: cho một cactus, hãy tìm kích thước tập độc lập lớn nhất.

Cách trực tiếp bắt đầu từ cây DFS. Nếu bỏ qua các cạnh ngoài cây, ta có bài toán tập độc lập trên cây với trạng thái

$$f(i, j),$$

trong đó $j \in \{0, 1\}$ cho biết cha của i có được chọn hay không. Tuy nhiên tập độc lập của cây DFS chưa chắc là tập độc lập của cactus, vì còn phải thỏa các cạnh ngoài cây.

Do mỗi cạnh ngoài cây nằm trên một chu trình riêng, ta chỉ cần nhớ thêm trạng thái của đỉnh trên cùng của chu trình chứa i . Gọi $k \in \{0, 1\}$ là trạng thái của đỉnh có độ sâu nhỏ nhất trên chu trình đó, và đặt

$$f(i, j, k)$$

là kích thước tập độc lập lớn nhất trong cây con gốc i , khi cha của i có trạng thái j và đỉnh trên cùng của chu trình có trạng thái k . Khi i là đỉnh đáy của chu trình và $k = 1$, ta cấm chọn i . Nhờ vậy các ràng buộc ngoài cây được đưa về trạng thái cục bộ, và độ phức tạp vẫn tuyến tính theo $|V|$ hoặc $|E|$, vốn cùng bậc trong cactus.

1.3 Quy hoạch động trạng thái nén trên cây DFS

Cây DFS cũng hữu ích khi đồ thị gần là cây, tức $|E| - |V|$ nhỏ.

Đếm tô màu. Cho đồ thị liên thông vô hướng $G = (V, E)$, cần đếm số cách tô V bằng k màu sao cho hai đầu mỗi cạnh có màu khác nhau, lấy modulo p . Giả sử

$$|V| \leq 100, \quad |E| < |V| + 10.$$

Nếu G là cây với $n = |V|$, đáp án là

$$k(k-1)^{n-1}.$$

Với đồ thị có ít cạnh ngoài cây, dựng cây DFS. Vì số cạnh ngoài cây không quá 10, có thể nén trạng thái bằng cách ghi nhận quan hệ màu giữa các đầu mút phía trên của các cạnh ngoài cây. Bản chất là xét các phân hoạch của tối đa 10 phần tử; số phân hoạch là

$$\text{Bell}(10) = 115975.$$

Một cách cài đặt dễ hơn là liệt kê bên ngoài phân hoạch màu của các đầu mút đặc biệt, rồi bên trong chạy quy hoạch động trên cây.

Ta còn có thể giảm kích thước đồ thị trước khi DP. Với $n > 1$, mỗi lá trong cây đóng góp nhân tử $k - 1$, nên có thể xóa lá lặp lại. Sau đó nén các chuỗi đỉnh bậc 2 thành một cạnh có trọng số. Với một đường dài l , tiền xử lý hai giá trị a_l, b_l : số cách tô phần trong đường khi hai đầu cùng màu và khi hai đầu khác màu. Sau khi nén, mỗi cạnh mang hai trọng số tương ứng, và DP chỉ còn chạy trên phần lõi của đồ thị. Nếu mọi đỉnh còn lại có bậc ít nhất 3, từ $\sum_v \deg(v) = 2m \geq 3n$ suy ra $n \leq 2(m - n)$, nên độ phức tạp phụ thuộc chủ yếu vào số dư chu trình $m - n$.

Đếm tập độc lập kích thước k . Biến thể khác là đếm số tập độc lập kích thước k modulo p , khi $|E| - |V|$ nhỏ. Sau khi dựng cây DFS T , đặt

$$f(i, j, S)$$

là số cách chọn tập độc lập kích thước j trong cây con gốc i . Tập S lưu các ràng buộc do những cạnh ngoài cây phủ cạnh nối i với cha của nó: cụ thể là cha của i và các đầu mút phía trên liên quan có được chọn hay không. Nếu C_i là tập cạnh ngoài cây phủ cạnh cha của i , độ phức tạp có dạng

$$O\left(n^2 \sum_{v \in V} 2^{|C_v|}\right).$$

Cách biểu diễn này vẫn nhanh ngay cả khi $|E| - |V|$ không quá nhỏ, miễn là mỗi cạnh cây chỉ bị ít cạnh ngoài cây phủ.

2 Cây BFS

Bên cạnh cây DFS còn có cây BFS. Tính chất của nó không thuận lợi cho DP cây kiểu trên, vì cạnh ngoài cây không nhất thiết nối tổ tiên-hậu duệ. Tuy nhiên nó có tính chất phân lớp rất mạnh: nếu chia đỉnh theo độ sâu trong cây BFS, thì mọi cạnh ngoài cây chỉ nối hai đỉnh cùng lớp hoặc hai lớp kề nhau. Từ tưởng này có thể dùng trong một số quy hoạch động trên đồ thị phân lớp.

3 Cây tròn-vuông của cactus

3.1 Định nghĩa

Một cactus là đồ thị vô hướng liên thông mà mỗi cạnh thuộc không quá một chu trình đơn.

Với cactus $G = (V, E)$, **cây tròn-vuông** của nó là đồ thị

$$T = (V_T, E_T)$$

thỏa các điều kiện sau. Tập đỉnh được chia thành

$$V_T = R_T \cup S_T, \quad R_T = V, \quad R_T \cap S_T = \emptyset.$$

Các đỉnh trong R_T gọi là **đỉnh tròn**; chúng tương ứng với đỉnh gốc của cactus. Các đỉnh trong S_T gọi là **đỉnh vuông**; mỗi đỉnh vuông tương ứng với một chu trình đơn của cactus.

Nếu một cạnh $e \in E$ không thuộc chu trình nào, ta giữ cạnh đó trong E_T như một cạnh nối hai đỉnh tròn. Với mỗi chu trình đơn R , tạo một đỉnh vuông p_R , rồi nối p_R với mọi đỉnh tròn nằm trên R .

Vì sao đây là cây? Các cạnh không thuộc chu trình được giữ nguyên nên không phá liên thông. Mỗi chu trình được thay bằng một đỉnh vuông nối tới mọi đỉnh trên chu trình, nên tính liên thông vẫn được bảo toàn. Nếu cactus có $c = |E| - |V| + 1$ chu trình, thì

$$|V_T| = |V| + c = |E| + 1.$$

Mặt khác, một chu trình dài r có r cạnh trong cactus và cũng đóng góp r cạnh từ đỉnh vuông tới các đỉnh tròn trong cây tròn-vuông, nên tổng số cạnh vẫn là $|E|$. Vì đồ thị thu được liên thông và có $|V_T| = |E_T| + 1$, nó là một cây.

3.2 Xây dựng

Có thể xây dựng cây tròn-vuông bằng thuật toán Tarjan tìm thành phần song liên thông đỉnh:

1. chạy Tarjan từ một đỉnh bất kỳ;
2. với mỗi thành phần song liên thông là một chu trình, lấy các đỉnh khỏi ngăn xếp; thứ tự trong ngăn xếp chính là thứ tự trên chu trình, tạo đỉnh vuông và nối tới các đỉnh tròn tương ứng;
3. nếu một cạnh là cạnh cây không bị chu trình nào phủ, thêm trực tiếp cạnh tròn-tròn đó vào cây tròn-vuông.

Một số tính chất cơ bản:

- không có cạnh nối hai đỉnh vuông;
- cây tròn-vuông không phụ thuộc vào gốc DFS, ngoại trừ cách đánh số các đỉnh vuông;
- nếu đặt gốc r , thì **sub-cactus** của một đỉnh tròn p là các đỉnh tròn nằm trong cây con của p trên cây tròn-vuông.

4 Ứng dụng của cây tròn-vuông

4.1 DP trên cactus

BZOJ 4316: tập độc lập lớn nhất. Ta dùng trạng thái giống cây:

$$f(i, 0/1)$$

là tập độc lập lớn nhất trong sub-cactus của i , tùy i có được chọn hay không. Nếu cạnh là tròn-tròn, chuyển giống cây. Nếu gặp cạnh tròn-vuông, lấy toàn bộ các đỉnh trên chu trình tương ứng và chạy DP tập độc lập trên vòng. Cách này có ít chi tiết hơn so với xử lý trực tiếp bằng cây DFS, vì Tarjan đưa các đỉnh trên chu trình ra theo đúng thứ tự.

BZOJ 1023: đường kính cactus. Cần tìm giá trị lớn nhất của khoảng cách ngắn nhất giữa hai đỉnh. Với đỉnh tròn, xử lý như đường kính cây: lưu độ sâu lớn nhất và nhì đi xuống. Với đỉnh vuông, cần xét đường đi ngắn nhất dài nhất có LCA là chính chu trình đó. Trên một chu trình, hai hướng đi tạo hai ứng viên; có thể dùng hàng đợi đơn điệu để xử lý các giá trị cực đại trên vòng.

4.2 Truy vấn đường đi ngắn nhất

Bài BZOJ 2125 yêu cầu nhiều truy vấn khoảng cách ngắn nhất giữa hai đỉnh trong cactus, với $n, m, q \leq 10^6$. Hai đỉnh trong cactus có các đường đi đơn tạo thành một chuỗi gồm một số cạnh cây và một số chu trình; trên mỗi chu trình, đường ngắn nhất luôn chọn phía ngắn hơn.

Gán trọng số cho cây tròn-vuông như sau. Chọn một đỉnh tròn làm gốc. Mọi cạnh tròn-tròn giữ trọng số như trong đồ thị gốc. Với một cạnh tròn-vuông, nếu đó là cạnh nối đỉnh vuông với cha của nó thì đặt trọng số 0; nếu không, đặt trọng số bằng khoảng cách ngắn nhất từ đỉnh tròn đó đến cha của đỉnh vuông trên chu trình.

Khi đó nếu LCA của hai đỉnh là đỉnh tròn, khoảng cách ngắn nhất chính là khoảng cách có trọng số trên cây tròn-vuông. Nếu LCA là đỉnh vuông, còn phải xét riêng chu trình của đỉnh vuông: dùng HLD hoặc bảng nhảy để tìm hai nhánh của truy vấn trên chu trình, rồi cộng phía ngắn hơn giữa hai vị trí đó.

4.3 Virtual cactus

Tương tự virtual tree, ta có thể dựng **virtual cactus**. Trước hết cần một tính chất: nếu một cây $T = (S_T + R_T, E_T)$ thỏa không có cạnh nối hai đỉnh vuông, thì nó là một cây tròn-vuông hợp lệ của một cactus nào đó. Chứng minh có tính xây dựng: với mỗi đỉnh vuông, nối các đỉnh tròn kề nó thành một chu trình.

Nếu trực tiếp lấy virtual tree của cây tròn-vuông, có thể xuất hiện cạnh ảo nối hai đỉnh vuông. Để tránh điều đó, với mỗi cạnh ảo đi ra từ một đỉnh vuông, chèn thêm một đỉnh tròn biểu diễn vị trí trên chu trình mà nhánh đó rời khỏi. Kích thước vẫn là $O(|S|)$ với S là tập đỉnh quan trọng. Cây ảo thu được lại là một cây tròn-vuông hợp lệ, tương ứng với một virtual cactus.

UOJ 87. Cho cactus, mỗi truy vấn là một tập đỉnh; cần tìm khoảng cách ngắn nhất lớn nhất giữa hai đỉnh trong tập. Dựng virtual cây tròn-vuông, đặt trọng số cạnh ảo bằng khoảng cách trên cây gốc, rồi bài toán trở về dạng truy vấn đường kính trên cấu trúc của BZOJ 2125.

UOJ 189. Cho cactus, hỗ trợ sửa trọng số cạnh và cho một số đường đi dạng ngắn nhất hoặc dài nhất, cần tính tổng trọng số của hợp các đường đó. Với cây, có thể dùng virtual tree để biến hợp các đường thành tổng các cạnh được phủ. Với cactus, dựng virtual cây tròn-vuông; trên mỗi chu trình, các phần bị phủ là hợp của một số đoạn, có thể sắp xếp và cộng bằng tiền tố.

Để hỗ trợ sửa cạnh, có thể duy trì ba loại độ sâu:

- depTree: tổng trọng số các cạnh cây từ gốc;
- depMin: trên mỗi chu trình đi qua, lấy phía ngắn;
- depMax: trên mỗi chu trình đi qua, lấy phía dài.

Khi sửa cạnh cây, cộng hiệu vào cả cây con. Khi sửa cạnh trên chu trình lẻ, tách chu trình tại cha của đỉnh vuông; một phía ảnh hưởng tới depMin, phía còn lại ảnh hưởng tới depMax. Các cộng đoạn trên cây con có thể dùng cây Fenwick hoặc cây đoạn.

4.4 Phân rã và phân tách trên cactus

Với bài đếm số đường đi đơn độ dài i xuất phát từ đỉnh 1, có thể phân rã trọng tâm trên cây tròn-vuông. Nếu $g_p(z)$ là hàm sinh của các đường đi từ p xuống cây con, thì với một chu trình gồm các đỉnh

$$R_1, R_2, \dots, R_r$$

không kể gốc chu trình, đóng góp có dạng

$$\sum_{i=1}^r g_{R_i}(z)(z^i + z^{r-i+1}).$$

Các phép nhân đa thức ở đỉnh tròn có thể xử lý bằng FFT; ở đỉnh vuông, không nên nhân thô từng hạng vì bậc có thể lớn. Ta tách thành tổng các đa thức đã dịch:

$$\sum_{q \in S_g} G_q(z)z^{\text{id}_g(q)} + \sum_{q \in S_g} G_q(z)z^{r-\text{id}_g(q)+1},$$

nên chỉ cần cộng đa thức sau khi dịch bậc. Cách này đạt $O(n \log^2 n)$ trong phác thảo của slide.

Với các bài cần cập nhật đường từ một đỉnh về gốc trong cactus có toàn chu trình lẻ, ta có thể mô phỏng ý tưởng heavy-light decomposition trên cây tròn-vuông. Sub-cactus tương ứng với cây con, nên truy vấn đếm trong cây con giống bài cây. Phần khó là cập nhật đường ngắn nhất hoặc dài nhất trên một chuỗi nặng. Ta phân loại các đỉnh tròn trên chuỗi: đỉnh bắt buộc đi qua, đỉnh chỉ nằm trên đường ngắn nhất, và đỉnh chỉ nằm trên đường dài nhất. Mỗi loại được duy trì riêng bằng cây đoạn trên một thứ tự DFS được chỉnh lại cho đỉnh vuông: trước tiên đưa tất cả đỉnh tròn kề đỉnh vuông vào thứ tự, rồi mới đệ quy vào các cây con của chúng.

5 So sánh với cây co chu trình

Cây co chu trình và cây tròn-vuông đều biến cactus thành cây, và đều có thể dùng cho phân tách hoặc virtual tree. Khác biệt chính là:

- cây co chu trình là cây có gốc; cây tròn-vuông tự nhiên là cây không gốc;
- cây tròn-vuông giữ tương ứng một-một giữa đỉnh gốc và đỉnh tròn, trong khi cây co chu trình có thể làm mất thông tin vị trí trong chu trình;
- mọi cây tròn-vuông hợp lệ đều tương ứng với một cactus, còn nhiều cactus khác nhau có thể cho cùng một cây co chu trình;
- khi cần biết một đỉnh thuộc sub-cactus nào của một chu trình, cây tròn-vuông có thể dùng nhảy trên cây hoặc HLD trực tiếp.

Vì không có các đỉnh gốc, cây tròn-vuông thường có tính chất tốt hơn cho virtual cactus, phân rã và các bài cần bảo toàn vị trí trên chu trình.

6 Cây tròn-vuông tổng quát

Với một đồ thị vô hướng tổng quát, nếu ta tạo một đỉnh vuông cho mỗi thành phần song liên thông đỉnh rồi nối nó với mọi đỉnh trong thành phần đó, ta nhận được một cấu trúc tương tự, gọi là **cây tròn-vuông tổng quát**. Theo định lý tương đương ở trên, có thể xem mỗi thành phần song liên thông như một chu trình của một cactus tương đương, dù thứ tự trên “chu trình” là tùy chọn. Cách nhìn này giúp biến một số bài trên thành phần song liên thông thành bài trên cây.

Bài “Thương nhân”. Cho đồ thị có trọng số đỉnh. Mỗi truy vấn hỏi trên tất cả đường đi đơn giữa hai đỉnh, giá trị nhỏ nhất có thể gặp là bao nhiêu. Dựng cây tròn-vuông tổng quát và đặt trọng số của mỗi đỉnh vuông bằng trọng số nhỏ nhất của các đỉnh tròn kề nó. Khi đó truy vấn trở thành lấy min trên đường đi trong cây.

CodeChef SADPAIRS. Cho đồ thị có màu đỉnh; với mỗi đỉnh, hỏi nếu xóa đỉnh đó thì có bao nhiêu cặp đỉnh cùng màu bị tách rời. Xét riêng từng màu, dựng virtual tree trên cây tròn-vuông tổng quát. Mỗi cạnh ảo tương ứng với một đường trên cây thật và đóng góp

số đỉnh trong cây con $\times$ số đỉnh ngoài cây con;

có thể dùng hiệu trên cây để cộng đóng góp. Với LCA tuyến tính bằng RMQ và sắp xếp virtual tree bằng radix sort, tổng độ phức tạp có thể đạt tuyến tính theo kích thước đầu vào.

Kết luận

Các kỹ thuật trong tài liệu đều cùng một mục tiêu: đưa phần “không phải cây” của đồ thị về một số nút hoặc trạng thái nhỏ, rồi tận dụng các thuật toán quen thuộc trên cây. Cây DFS thích hợp cho đồ thị gần cây và các ràng buộc do cạnh ngoài cây phủ đường đi. Cây BFS thích hợp cho phân lớp. Cây tròn-vuông và biến thể tổng quát là công cụ mạnh cho cactus và thành phần song liên thông, đặc biệt khi cần virtual tree, truy vấn đường đi, phân rã hoặc heavy-light decomposition.